

Enhanced fingerprint pattern classification: Integrating attention modules with lightweight deep learning models

Esther Mukoya  | Richard Rimiru | Michael Kimwele

School of Computing and Information Technology, Jomo Kenyatta University of Agriculture and Technology, Nairobi, Kenya

Correspondence

Esther Mukoya, School of Computing and Information Technology, Jomo Kenyatta University of Agriculture and Technology, P. O. Box 62000, 00200 Nairobi, Kenya.
Email: emgwada@jkuat.ac.ke

Abstract

Large fingerprint databases can make the automated search process tedious and time-consuming. Fingerprint pattern classification is a significant step in the identification system's complexity in terms of time and speed. Although several fingerprint algorithms have been developed for classification tasks, further improvements in performance and efficiency are still required. Most of the fingerprint algorithms use deep learning techniques. However, some deep learning techniques can be resource-intensive and computationally expensive, while others can disregard the spatial relationships between the features used in classifying fingerprint patterns. This study proposes using lightweight deep learning models (i.e., MobileNet and EfficientNet-B0) integrated with attention modules to classify fingerprint patterns. The two lightweight models are modified, yielding MobileNet+ and EfficientNet-B0+ models. The lightweight deep learning models can help achieve optimal performance and reduce computational complexity. The attention modules focus on distinctive features for classification. Our proposed approach integrates four attention modules for fingerprint pattern classification into two lightweight deep learning models, that is, MobileNet+ and EfficientNet-B0+. To evaluate our approach, we use two publicly available fingerprint datasets, that is, the NIST special database 301 dataset and the LivDet dataset. The evaluation results show that the EfficientNet-B0+ model achieves the highest classification accuracy of 97% with only 854,086 training parameters. As a conclusion, we consider the training parameters small enough for the EfficientNet-B0+ model to be deployed on low-resource devices.

KEYWORDS

attention modules, EfficientNet-B0, fingerprint classification, lightweight models, MobileNet

1 | INTRODUCTION

Deep learning models have rapidly developed in different fields, such as image classification,^{1–3} multitask learning,⁴ semantic segmentation,⁵ image retrieval, identification of persons,⁶ and biometric recognition, for example, fingerprint pattern recognition. Fingerprint pattern recognition serves both verification and identification purposes. Identification involves selecting a single fingerprint image from a gallery of fingerprint images^{7,8} while verification is used to compare

This is an open access article under the terms of the Creative Commons Attribution License, which permits use, distribution and reproduction in any medium, provided the original work is properly cited.

© 2024 The Authors. *Engineering Reports* published by John Wiley & Sons Ltd.

two fingerprints and determine if they belong to the same individual. Given the growth of fingerprint databases, the speed and accuracy of fingerprint identification and verification have become increasingly important.

Hence, fingerprint pattern classification plays a significant role in ensuring an efficient identification and verification process. As such, numerous studies have explored fingerprint pattern classification using deep learning models. For instance, Bhavesh et al.⁸ proposed a deep learning convolutional neural network (CNN) consisting of a preprocessing module and a deep CNN for classification. Additionally, Shervin et al.⁹ applied transfer learning using a pretrained ResNet50 model¹⁰ to classify fingerprints.

Despite the progress in fingerprint pattern classification, there are still some inherent limitations. For example, classifying fingerprint patterns using deep learning models with many layers requires high computational resources. Additionally, using many parameters in deep learning models can cause over-fitting of the models and poor generalization of new data. To address the above issues, lightweight deep learning models have been proposed and developed in the literature.^{7,11,12} The lightweight deep learning models use fewer layers, making them less complex for classification tasks. For instance, experiments performed by Li et al.¹¹ demonstrated using the squeeze and excitation (SENet) module lightweight deep learning model for fingerprint pattern classification. Similarly, Saeed et al.¹² designed a lightweight deep learning model for automatic fingerprint pattern classification. Their model optimizes the CNNs structure by determining the number of filters and layers needed to classify fingerprints using cross-sensor fingerprints. Lastly, Jian et al.⁷ used lightweight deep learning models to classify fingerprints using singularity features from the fingerprint images. While lightweight deep learning models can help reduce complexity during model training, they can suffer from reduced performance accuracy compared to full-depth deep learning models due to information loss.¹³ Hence, some lightweight deep learning models combine feature discrimination methods to boost performance. The feature discrimination methods include enhancement,^{14,15} fingerprint restoration^{16,17} and center loss.¹⁸ However, the lightweight deep learning models still suffer the limitations of feature refinement.

1.1 | Contributions

This study proposes a hybrid method for classifying fingerprint image patterns. The hybrid method modifies two lightweight deep learning models to yield EfficientNet-B0+ and MobileNet+ models with attention modules. The attention modules employ a mechanism that can learn the most significant details from an input image.^{19–22} This can improve the performance of image classification tasks. By integrating attention modules into lightweight learning models, the model can focus more on the significant features for refinement.¹⁹ This allows the model to memorize only the significant features and use them for classification. We performed experiments using four different attention modules combined with the two lightweight deep learning models to enhance the learning ability of the models in fingerprint pattern classification. In addition, we performed experimental analysis to determine the appropriate model layer for adding the attention module. More concretely, this study makes the following contributions. First, the study proposes using two different pretrained lightweight deep learning models to classify fingerprint patterns into six classes. Secondly, the study enhances the performance of the pretrained models by incorporating four different attention modules. The appropriate layer number is determined experimentally. Lastly, the study evaluates the two models based on accuracy, precision, and recall and presents the results.

The remainder of this paper is organized as follows. Section 2 presents the background of this study. Section 3 presents our proposed approach. Section 4 presents the materials and methods used in this study. Section 5 presents the experimental results. Lastly, Section 6 discusses the evaluation results and presents the conclusion.

2 | RELATED WORKS

Lightweight deep learning models combined with attention modules have witnessed significant growth.^{11,23–26} In this section, we perform a state-of-the-art analysis to give the background and motivation for our work. The section is organized into four parts. The first part explains the background of fingerprint pattern classification. The second part explains the background of lightweight deep learning models. The third part explains the background of attention modules. Lastly, the fourth part explains the application of attention modules. We use this background to motivate the work we present in this paper.

2.1 | Fingerprint pattern classification

The task of fingerprint pattern classification aims to improve the efficiency of identifying fingerprints by offering an indexing mechanism. The indexing mechanism reduces search complexity in large fingerprint databases, resulting in faster retrieval times.²³ In this regard, different methods have been devised and proposed in the literature for classifying fingerprint patterns. For instance, Galton et al.²⁷ devised a method for classifying fingerprint patterns based on their characteristics to form classes.²⁸ The classes formed include plain arch, tented arch, whorl, left loop, right loop, and double loop. The classes are formed by locating the significant points in fingerprints, counting them, and evaluating the core-delta pairs to place them in a class.²⁹

2.2 | Lightweight deep learning models

Lightweight deep learning models are implemented using CNNs. The CNNs are a type of deep neural network that are specifically designed for processing image tasks. Most of the tasks that use the above neural networks utilize pretrained CNNs. The pretrained CNNs refer to neural networks that undergo training on a large dataset before being used for a specific task, for example, the LeNet.^{25,30} The pretrained CNNs are both lightweight and deep. The lightweight models contain fewer layers, making them computationally cheaper than deep learning models. Using pretrained CNN models improves the model's speed and accuracy during model training. Several studies in the literature have used lightweight models to classify fingerprints. For instance, Gan et al.¹¹ use the SqueezeNet model to classify fingerprints, Michele et al.³¹ use the MobileNet model to recognize palm prints, and lastly, Hsiao et al.³² use the EfficientNet model for iris recognition.

In the above examples, the MobileNet and EfficientNet-B0 models were trained on the ImageNet dataset. The EfficientNet-B0 model uses a new scaling method that balances the depth, width, and resolution of CNNs.³³ The training aimed at reducing the computational complexity without affecting performance in the classification and recognition tasks. For example, the EfficientNet-B0 model balances the depth, width, and resolution of the neural network to obtain good performance with fewer parameters.³⁴ MobileNet substitutes convolutional layers with depth-wise separable convolutional blocks. The lightweight deep learning modules can help improve feature representation during classification tasks when integrated with attention modules.

2.3 | Attention modules

Integrating attention modules into lightweight deep learning models allows them to focus on the most critical aspects of the image rather than the global features.²⁶ The attention module can be regarded as a tool that can learn the most significant features of an image. Hence, the attention module can be adapted for image classification tasks.¹⁹ As a general observation, attention modules are implemented with a gating function, for example, softmax sigmoid with sequential or parallel techniques.

Several studies in the literature have explored integrating attention modules into deep learning models. For example, Wang et al.³⁵ developed a residual attention network that stacks several attention modules. Different attention modules have been proposed in the literature, such as the convolutional block attention module (CBAM).³⁶ The CBAM module consists of two independent submodules, that is, the channel attention submodule and the spatial attention submodule. The channel attention submodule performs attention mechanisms on a channel. The spatial attention submodule performs attention mechanisms on the feature maps generated between the channel attention submodule and the convolutional layer. The channel attention submodule utilizes max and average-pooling outputs with a shared network. Average pooling is used to learn the target image's extent effectively. On the other hand, the max-pooling helps to infer finer channel-wise attention. The spatial submodule utilizes two similar outputs pooled along the channel axis to a convolution layer. The spatial attention submodule is complementary to the channel attention submodule as it focuses on the informative point of the image.

The study by Hu et al.³⁷ designed the SENet attention module that learns the interchannel relationships of convolutional layers. In the SENet attention module, a global average pooling mechanism compresses the spatial features and computes the channel-wise attention features. The SENet module consists of a convolutional block that indicates the number of channels, the average pooling layer, a dense layer with a rectified linear unit (ReLU), another dense layer, and a sigmoid activation function. The module first squeezes the features using global average pooling on each input channel.

Then, an excitation module achieved by the two dense layers (i.e., ReLU and the sigmoid function) uses the output from the SENet operation to capture all the channel dependencies. Finally, the excitation channel is attained by multiplying all the features mapped with the input channels.

Other kinds of attention modules (e.g., the self-attention and the dual attention module) have been used in the literature to improve the performance of deep learning models.^{38,39} The self-attention module consists of two parallel convolutional layers, the softmax layer, and the ReLU activation layer. Fu et al.³⁹ adopted the dual attention module for scene segmentation. Dual attention module consists of two attention submodules, that is, the position attention and channel attention submodules. The position attention submodule encodes a broader range of contextual information into local features. The channel attention submodule explicitly models the interrelationships between channels.

2.4 | Application of attention modules

Many recent biometric studies have incorporated attention modules into their classification tasks. For example, Zhang et al.²² combined a CBAM with a lightweight deep learning model for finger vein recognition. The attention module focuses on the most informative features, making them more discriminative. Ling et al.²¹ developed an attention-based deep learning model for face biometric recognition. The attention module was embedded to refine the features extracted by the model, making the components of the face feature map more distinguishable. Other than biometrics, CBAM has also been used in Covid-19 diagnosis^{40,41} to boost the performance of CNNs in classification processes.

Based on the above background, we consider attention modules essential components and a good motivation to incorporate them into lightweight deep learning models for fingerprint pattern classification. In the subsequent section, we describe the materials and methods that we apply in our approach.

3 | PROPOSED APPROACH

As previously mentioned, our proposed approach uses two lightweight deep learning models, that is, the EfficientNet-B0 and the MobileNet models. We first modify the two models to yield what we call in this study the EfficientNet-B0+ and the MobileNet+ models. The MobileNet+ includes depthwise separable convolutions, pointwise convolutions, inverted residuals, and linear bottlenecks. The depthwise separable convolutions filter each channel separately to extract features and capture spatial information while minimizing computational complexity. The pointwise convolution follows suit, allowing the model to capture more complex and abstract features. The linear bottlenecks and inverse residuals prevent information loss in low-dimensional feature maps and improve network accuracy.

The MobileNet+ model was designed as illustrated in Figure 1. The adaptive average pooling layer replaces the average pooling. Batch normalization and softmax layers are included. The batch normalization layer helps to train the network faster. The layer smooths out the optimization process, ensuring a consistent and stable gradient behavior, which leads to faster training. The adaptive average pooling layer helps to reduce the number of parameters. The softmax layer is used for classification. After every batch normalisation layer, the ReLU activation layer is computed. A ReLU layer operating in the convolutional layer may lead to information loss that can affect the model prediction ability. Hence, we improve the model by adding an attention module after the InvertedResidual.

The EfficientNet-B0+ model architecture includes a mobile inverted convolutional (MBconv) layer as the basic building block. The MBConv is used for feature extraction. A SENet attention module is automatically embedded in the EfficientNet-B0 model for improved feature representation. The EfficientNet-B0+ also employs the *swish* activation function, whose performance exceeds that of ReLU.⁴² The swish is a nonlinearity function computed as a product of sigmoid and linear activations. Unlike ReLU, the swish function does not nullify negative values and has faster training times, especially when approaching non-zero gradients. The swish activation function achieves smoothness, nonmonotonicity and one-sided boundedness at zero. The smoothness leads to better convergence. The model learns more complex patterns from the data using nonmonotonicity since it does not follow a single direction.

ReLU performs piecewise linear activation and is monotonically increasing. The EfficientNet-B0+ model had the attention module integrated at MBconv7. Figure 2 shows the architecture of the EfficientNet-B0+ used in this study. Figure A1 shows the implementation details for this model. The implementation of the MobileNet+ model follows a similar approach.

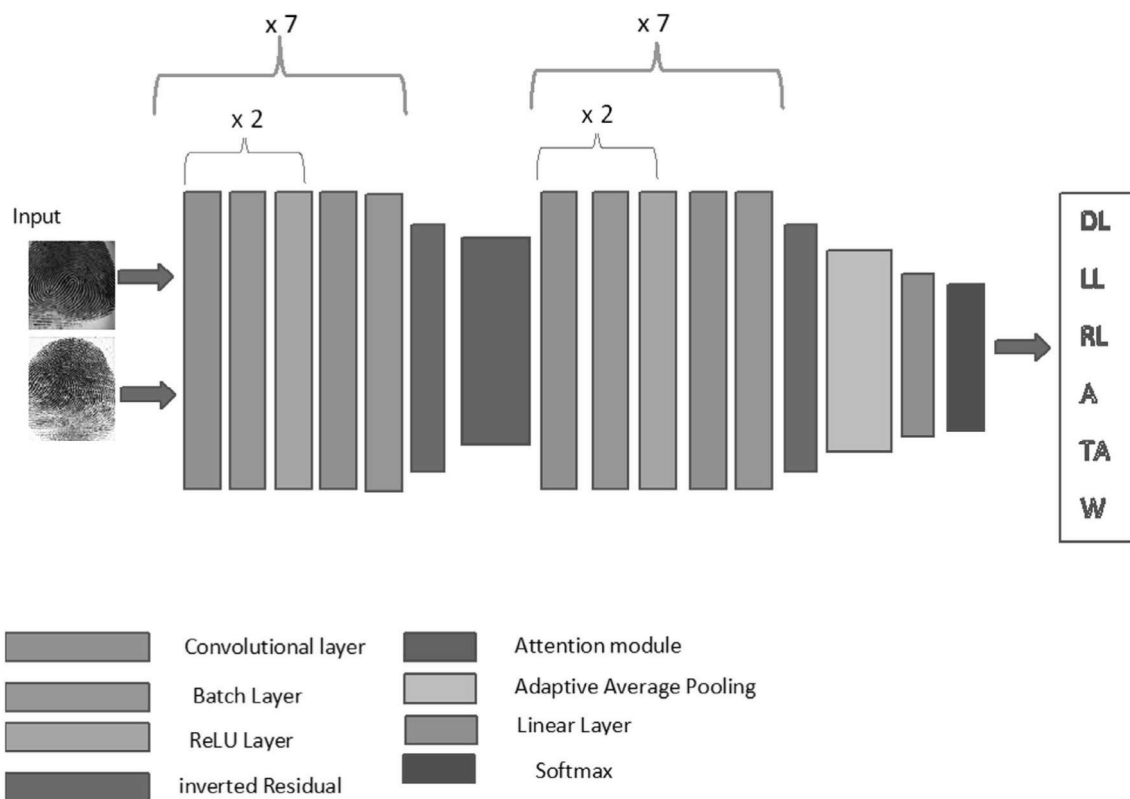


FIGURE 1 Architecture of the MobileNet+ lightweight deep learning model used in this study.

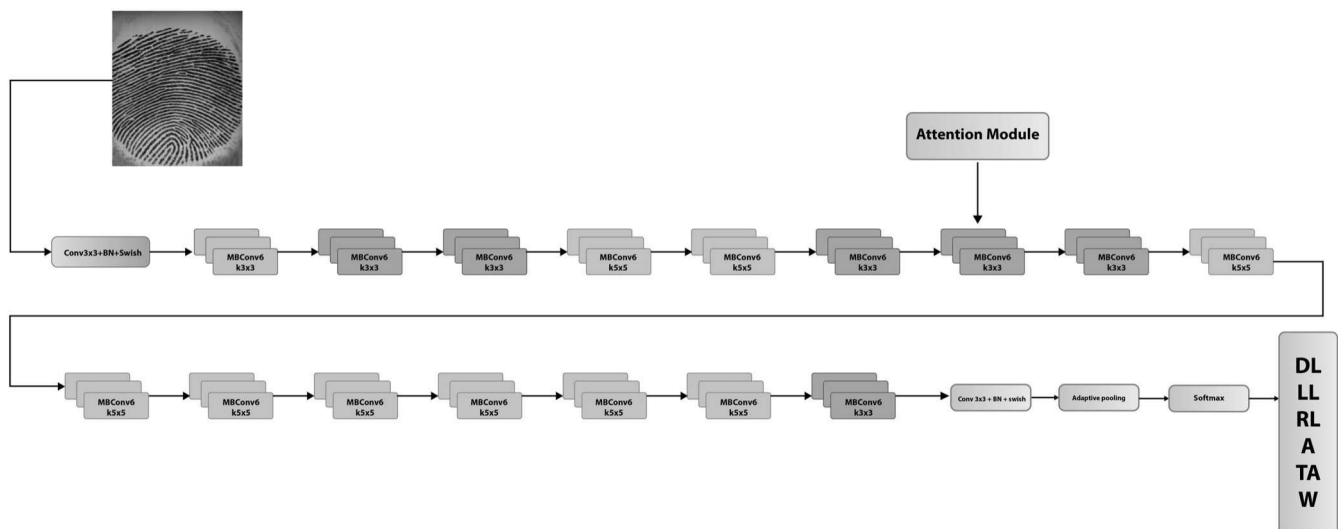


FIGURE 2 EfficientNet-B0+ model architecture integrated with attention module.

4 | MATERIALS AND METHODS

This section presents the materials used in this study and the methods followed to perform experiments. The section begins by describing the datasets used in performing experiments. A description of the experimental setting follows this. The section then describes the approach presented in this paper. Lastly, the section explains how the models used in our proposed approach were trained and how their testing was done.

TABLE 1 Details of the LiveDet dataset.

Dataset	Sensor used	Size of image
LivDet-2015	Biometrika	1000 × 1000
	Digital Persona	252 × 324
	GreenBit	500 × 500
	Cross Match	640 × 480
LivDet-2013	Biometrika	315 × 372
	Ital data	640 × 480
LivDet-2011	Biometrika	312 × 372
	Digital Persona	355 × 391
	Ital data	640 × 480
	Sagem	352 × 384
LivDet-2009	Biometrika	312 × 372
	CrossMatch	640 × 480
	Identix	720 × 720

4.1 | Dataset

The study combines two public fingerprint datasets. The first dataset was obtained from the National Institute of Standards and Technology Special Database, that is, NIST SD 301 dataset.⁴³ The second dataset is the LivDet competition dataset. Combining the two datasets yields 8400 fingerprint images. Hence, the study considers the two datasets comprehensive with sufficient fingerprint images for classification.

The images in the different LivDet datasets were collected using different numbers of sensors, that is, three fingerprint sensors for the LivDet-2009 dataset, four for the LivDet-2011 dataset and two fingerprint sensors for the LivDet-2013 and four for the LivDet-2015 dataset. Table 1 shows the details of the images from the LivDet datasets.

4.2 | Dataset preprocessing

The fingerprints were manually annotated via visual inspection into six classes: (1) the plain arch, (2) the double loop, (3) the left loop, (4) the right loop, (5) the tented arch, and lastly, (6) the whorl. The annotation into the above six classes was based on established fingerprint pattern definitions.⁴⁴ The annotation followed the number and location of significant points and ridge flows.²⁹ The plain arch-type fingerprint images have ridges that rise to a bump from one side and exit from the opposite side. Similar to the plain arch type is the tented arch, with one or more ridges exhibiting a high curvature. The loop type has ridges that enter and exit in the same direction, while the whorl has circular ridges around the centre of the image.

The tented arch and loop fingerprint patterns have a single core and delta point. Loops are divided into two classes. The first class is the *left loops* with ridges that enter and leave from the left side. The second class is the *right loops* with ridges that enter and leave from the right side. The whorl fingerprint pattern contains one or two cores and two delta points, while the double loop has two cores and two delta points. To ensure that no fingerprint images were misannotated, the techniques employed by Iwamoto⁴⁵ were used to cross-check the fingerprint images in the dataset. These techniques were used to describe the different fingerprint patterns based on the global ridge structure and the major points, that is, the core, delta, and loop points. Note that the fingerprints of the plain arch class do not have the core and the delta points.

This study used six different fingerprint classes, as shown in Figure 3. The figure depicts the arch class, which can be further classified into two main groups: the plain arch or simply arch and the tented arch. These two groups together make up 5% of the world's population.⁴⁶ The right and left loops make up approximately 60% to 75% of fingerprints worldwide.⁴⁶ Additionally, there is the double loop, also known as the twin loop, which combines the left and right loop patterns. Finally, the whorl fingerprint pattern accounts for approximately 25%–35% of fingerprints worldwide.⁴⁶

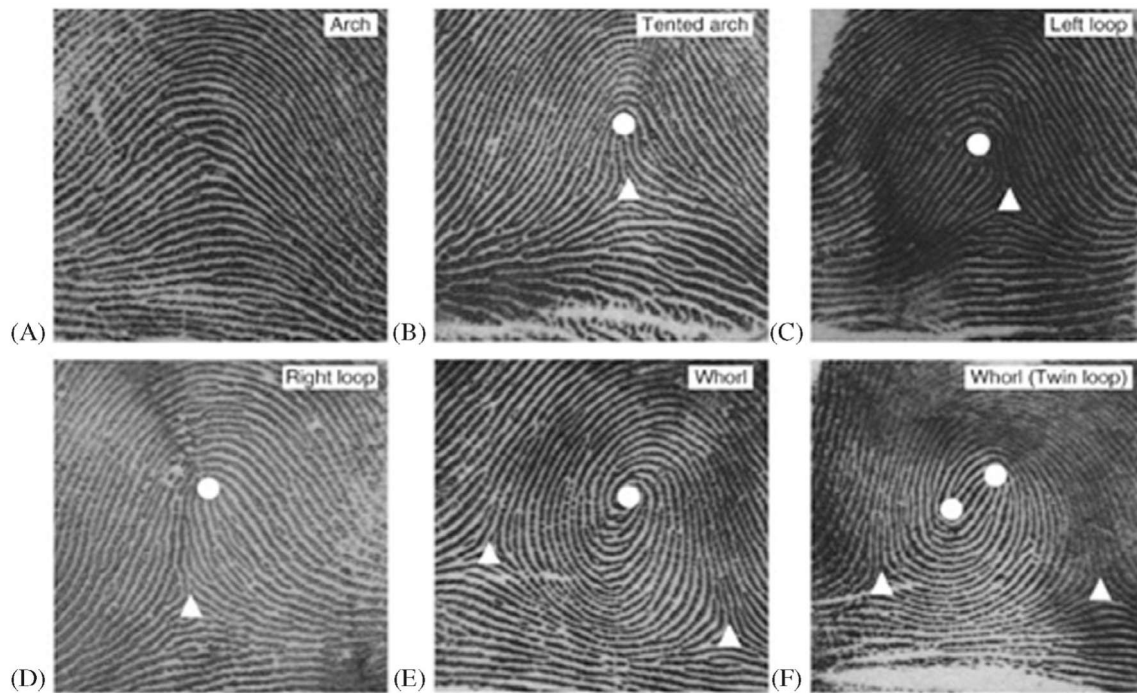


FIGURE 3 Illustration of the different types of fingerprint patterns.

```
class MyDataset(Dataset):
    def __init__(self, data, targets, transform=None):
        self.data = data
        self.targets = targets
        print(self.data[1])
    def __getitem__(self, index):
        x = cv2.imread(self.data[index], cv2.IMREAD_COLOR)
        #Apply data augmentation
        if self.transform:
            x = self.transform(x)
        #Normalize and preprocess the image
        x = x / 255.
        x = cv2.resize(x, (128,128))
        x = np.transpose(x, (2, 0, 1))
        y = self.targets[index]
        return np.float32(x), np.longlong(y)
    def __len__(self):
        return len(self.data)
# Define data augmentation
data_transform = transforms.Compose([
    transforms.ToPILImage(),
    transforms.RandomRotation(degrees=40),
    transforms.RandomHorizontalFlip(),
    transforms.RandomVerticalFlip(),
    transforms.ToTensor(),
])
```

FIGURE 4 Implementation for preprocessing the data.

Figure 4 shows the implementation for preprocessing the datasets. Lines 1-6 define the class, the data, and the class labels (targets). Line 7 reads in the data, and lines 9 and 10 apply the specified augmentation. Line 12 is used for normalization of the data. We resize the images on line 13 and perform a transpose on line 14. The transpose is used here to improve the resolution of the images for accurate predictions. Line 15 is used to retrieve the labels of the images in the dataset. The index refers to the index of the current input image in the dataset. Line 16 converts the input data type x into a 32-bit float data type, while the class labels y are converted into a 64-bit integer suitable for representing classes. Lines 17 and 18 return the size of the dataset in the experiment. The augmentation code function is defined in lines 20–26.

4.3 | EXPERIMENTAL SETTING

To run the experiments, the dataset was divided into training and test sets in the ratio of 80:20, respectively. Data augmentation was performed in the training dataset to increase the number of images to achieve good performance in our model. We used common data augmentation techniques in all our experiments, including random rotation and horizontal flip. Similar to the study by,⁴⁷ this study adheres to the deep learning principle of utilizing a large dataset differs from the pre-trained model dataset. The convolutional base performed feature extraction, while the output layer and the classifier helped predict the features. The experimental setting used EfficientNet-B0+ and MobileNet+ as the lightweight deep learning models. Attention modules were integrated into the models. The experiments performed were used to examine different positions for the integrated attention modules, specifically at blocks seven, eight, and nine of the EfficientNet-B0+ and MobileNet+ models. However, on blocks eight and nine the feature maps overlap highly. Integrating attention modules here may not be effective.

The MobileNet+ model typically consists of multiple convolution neural network blocks. As the model progresses, the convolution neural network blocks in MobileNet+ gradually increase the learnt feature maps. Integrating the attention modules at these blocks to highlight the significant features is essential. This allows the models to focus on high-level features typically achieved from block seven onwards.

The EfficientNet-B0+ baseline architecture comprises several mobile inverted bottleneck convolutions (MBConv) with varying kernel sizes, feature map expansion phases and reduction ratios. As we go deeper into the model from block seven onwards, the MBConv includes a skip connection and dropout layer integration. Furthermore, these MBConv blocks have larger feature maps. Integrating the attention modules at these blocks allows the model to work on a wider feature map than earlier layers. It was observed that integrating the attention module at block seven provided the best classification accuracy.

4.4 | Training and testing details

The algorithms were implemented using PyTorch framework. The model training was conducted on a standard laptop with an Intel Core i7 3.8 2.3 GHz CPU. The different models integrated with attention modules were trained, validated, and tested using the same dataset and hyperparameter training options. The Adams optimizer hyperparameter was used due to its effective convergence. The learning rate was set to 0.0001. The region of interest (ROI) was extracted from the images, the pictures normalized, and the input size was set to 128×128 with three channels. Cross-entropy loss function was used to measure the quality of a network's prediction results. The batch size was set to 4, and the model trained for 20 epochs. The dataset was split randomly into 80% for training and 20% for testing. The network training parameter details are shown Table 2.

A total of eight different experiments were conducted where the two lightweight learning models were each integrated with the four different attention modules. The two lightweight learning models integrated the attention modules at the same location. Standard evaluation metrics, such as confusion matrix, precision, recall, accuracy, and F1 score, were employed to evaluate the performance of the models. The recall metric measures the number of positive class predictions made with all positive predictions. Precision is a measure of accurately predicted true positives to the total number of positively predicted observations, whether correctly or incorrectly. The F1 score balances both the prediction and recall metrics. The validation accuracy computes the efficiency of the models during training to determine the effectiveness of each trained epoch. Equations (1), (2), (3), and (4) show the recall, precision, F1 score, and accuracy computations, respectively.

TABLE 2 Network training parameters.

Parameter name	Parameter description
Training set: Test set	80:20
Optimizer	Adams
Batch size	4
Learning rate	0.0001
Loss function	Cross-entropy loss

```

def train(model, loader, optimizer, loss_fn, device):
    epoch_loss = 0
    acc_list = []
    counter = 0
    model.train()
    for i, (x, y) in enumerate(loader):
        x = x.to(device, dtype=torch.float32)
        y = y.to(device, dtype = torch.long)

        optimizer.zero_grad()
        yp = model(x)

        loss = loss_fn(yp, y)
        acc = accuracy(yp, y, x.shape[0])
        acc_list.append(acc)
        loss.backward()
        optimizer.step()
        epoch_loss += loss.item()

    epoch_loss = epoch_loss/len(loader)
    acc_final = sum(acc_list) / len(loader)
    return epoch_loss, acc_final

```

FIGURE 5 Implementation for training the models.

$$\eta = \frac{\mu}{\mu + \tau}, \quad (1)$$

where η is the recall, μ is the true positive and τ is the false negative.

$$\psi = \frac{\mu}{\mu + \nu}, \quad (2)$$

where ψ is the precision and ν is false positive

$$\phi = 2 \times \frac{\psi \times \eta}{\psi + \eta}, \quad (3)$$

where ϕ is the F1 score.

$$\kappa = \frac{\mu + \tau}{\mu + \xi + \nu + \tau}, \quad (4)$$

where κ is the accuracy and ξ is the true negative.

Figure 5 shows the implementation for training the models. Lines 2–4 are for initialization. Line 2 initializes the epoch variable to store cumulative epoch loss. Line 3 initializes the accuracy list to store the accuracy for each batch. Line 4 has counter 0 that initializes the counter for tracking. Line 5 sets the model training mode. We then iterate over the dataset (loader) to provide batches of input data x and their corresponding labels y in line 6. Lines 7 and 8 move the input data and the labels into specified devices, respectively. The two lines also convert them to appropriate data types for training. The gradients of all optimized torch tensors are cleared at line 10. Line 11 computes the predicted values yp , while line 13 adds the loss between the expected outputs, yp and the labels, y . The accuracy is computed and added to the accuracy list on lines 14 and 15, respectively. On lines 16 and 17, the loss gradient is computed for the parameters, and the model's parameters are updated using the optimizer. Line 18 shows the accumulation of loss on the current batch. Lines 20 and 21 give the final average loss and final average accuracy, respectively. Lastly, line 22 returns the final epoch loss and accuracy.

5 | EVALUATION RESULTS

This section presents the results of the experiments performed to evaluate our proposed approach. The results are presented in two parts. The first part presents the results for the MobileNet+ model, and the second part presents the results for the EfficientNet-B0+ model.

5.1 | MobileNet+ model with attention modules

Table 3 shows the results of the four attention modules when integrated into the MobileNet+ model. The dual attention module recorded the best accuracy of 96.8%, precision of 96.7%, recall of 96.8%, and F1 score of 96.7%.

The accuracies of the different fingerprint patterns ranged from 97% to 99%, with the whorl pattern exhibiting the best accuracy. The whorl pattern is consistent and has unique geometric characteristics that make it distinctive. The tented arch pattern accuracy was poor compared to other fingerprint patterns. This can be attributed to its similarity with the left and right loop patterns. The model had 307,590 training parameters, making it lighter and less complex. The reduced complexity allows for faster inference times and quicker response time when integrated into real-time systems.

Figure 6A–D shows the training and validation accuracy for the four attention modules integrated into the MobileNet+ model. The obtained model training and validation curves are all smooth with minimal fluctuations. From the graphs,

TABLE 3 Results achieved for each of the attention modules in MobileNet+ model.

Attention module	Accuracy %	Precision %	Recall %	F1 Score %
Squeeze and excitation	95.8	95.7	95.8	95.7
Self	96.2	96.1	96.2	96.1
CBAM	96.6	96.6	96.7	96.6
Dual	96.8	96.7	96.8	96.7

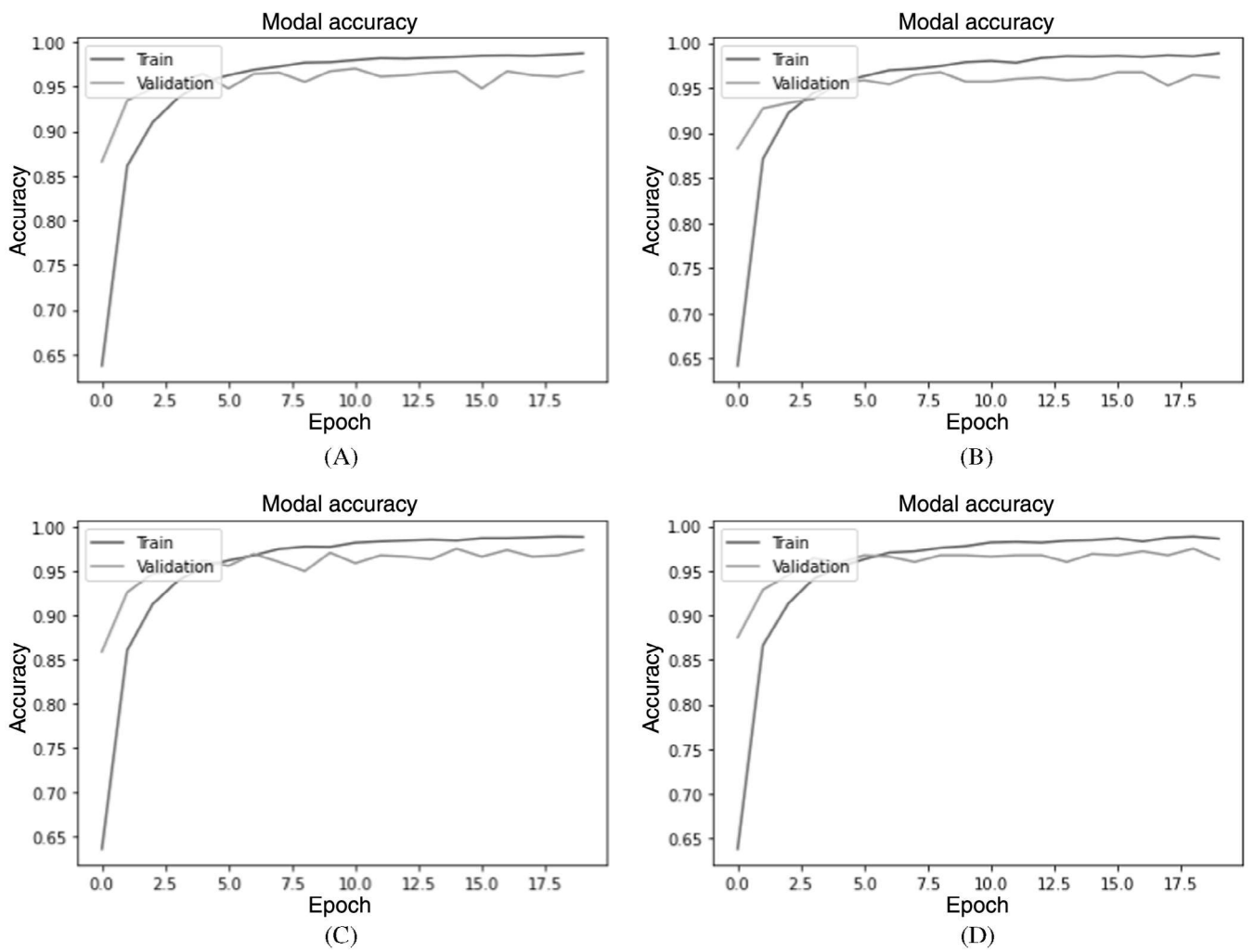


FIGURE 6 Illustration of the MobileNet+ model results. (A) shows squeeze and excitation attention module accuracy, (B) shows self-attention module accuracy, (C) shows the convolutional block attention module (CBAM) attention module accuracy graph, (D) shows dual module accuracy.

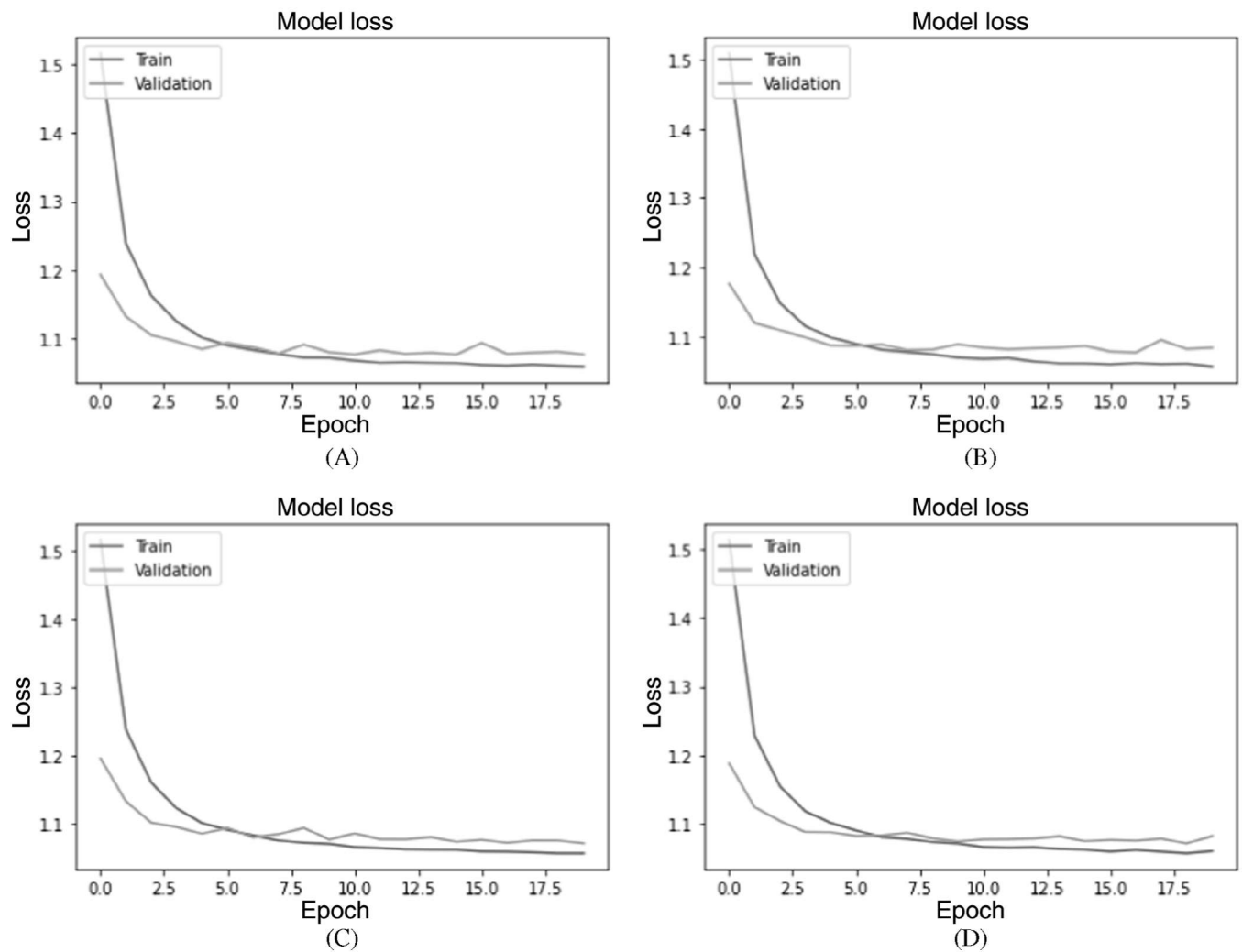


FIGURE 7 Illustration of the MobileNet+ model results. (A) shows squeeze and excitation attention module loss graph, (B) shows self-attention module loss, (C) shows the convolutional block attention module (CBAM) attention module loss graph and (D) shows dual attention module loss.

the training and validation curves are close and do not show any overfitting. The validation accuracy lines oscillate from 0.86 to 0.95 for SENet, 0.86 to 0.96 for self-attention, 0.86 to 0.96 for CBAM, and 0.86 to 0.97 for dual attention module.

Figure 7A-D shows the training and validation loss graphs for the four attention modules integrated into the MobileNet+ model. Similar to the accuracy graphs, the loss graphs exhibit minimal fluctuations, indicating that the model is not overfitting.

To provide a visual interpretation that assesses the performance of each attention module for the MobileNet+ model, confusion matrices are showcased using the test dataset as shown in Figure 8A–D. The first class (i.e., class 0) represents the plain arch, class 1 represents the left loop, class 2 represents the right loop, class 3 represents the whorl, class 4 represents the tented arch and class 5 illustrates the double loop. The values on the diagonals of the confusion matrix indicate the number of correct classifications for each category. We observe that the recognition accuracy differs for the different classes with the whorl class giving the highest number of correct classifications.

5.2 | EfficientNet-B0+ model with attention module

Table 4 shows the results of the four attention modules when integrated into the EfficientNetB0+ model. The dual attention module achieved the best performance with a training accuracy of 97%, precision of 96.8%, recall of 97%, and F1 score

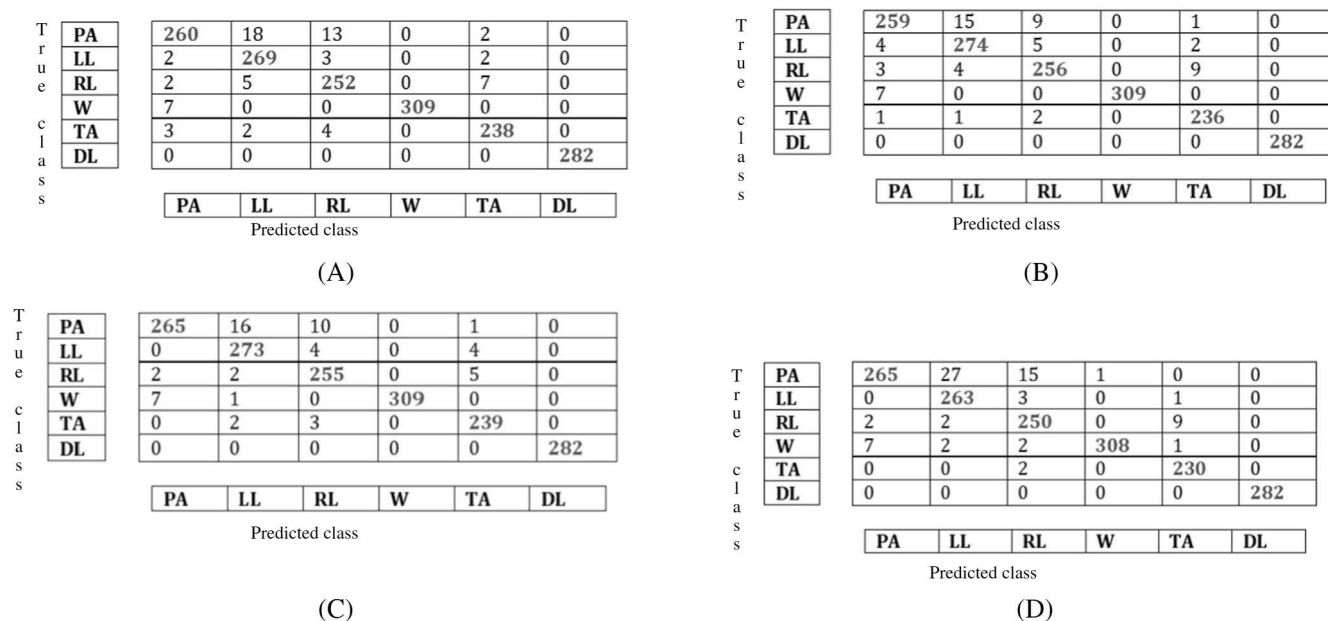


FIGURE 8 Illustration of the MobileNet+ model results. (A) shows the squeeze and excitation attention module confusion matrix, (B) shows the self-attention module confusion matrix, (C) shows the convolutional block attention module (CBAM) attention module confusion matrix and (D) shows the dual attention module confusion matrix.

TABLE 4 Results achieved for each of the attention modules in the EfficientNet-B0+ model.

Attention module	Accuracy %	Precision %	Recall %	F1 score %
Squeeze and excitation	96.8	96.7	96.8	96.8
Self	96.4	96.3	96.4	96.3
Convolutional block attention module	96.7	96.6	96.7	96.6
Dual	97	96.8	97	96.9

of 96.9%. The accuracies of the different fingerprint patterns ranged from 87% to 99%, with the whorl pattern exhibiting the best accuracy.

The model had 805,086 training parameters, making it lighter and less complex. The EfficientNet-B0+ model uses the MBconv of MobileNet+ as the backbone. It also uses the SENet network for its structure optimization. Compared to the other attention modules, the dual attention module performed best when integrated with the EfficientNet-B0+ model.

Figure 9A–D shows the training and validation accuracy for attention modules integrated into the EfficientNet-B0+ model. The model training and validation curves were all smooth with minimal fluctuations. The graphs show that the training and validation curves are even with no overfitting. The validation accuracy lines oscillate from 0.88 to 0.96 for SENet module, 0.88 to 0.95 for self-attention, 0.89 to 0.96 for CBAM, and 0.89 to 0.97 for dual attention. The dual attention module has a higher average accuracy level than the others.

Figure 10A–D shows the training and validation losses for the four attention modules integrated into the EfficientNet-B0+ model. The graphs show that the training and validation curves are even with no overfitting.

Figure 11A–D shows how well the models classify different fingerprint patterns using confusion matrices from the test dataset. The values on the diagonal of the confusion matrix show the number of accurate classifications for each category.

6 | DISCUSSION AND CONCLUSION

Fingerprint pattern classification has existed for many years, and several advanced algorithms have been proposed and tested. This study demonstrates two lightweight deep learning models, MobileNet and EfficientNet-B0, integrated

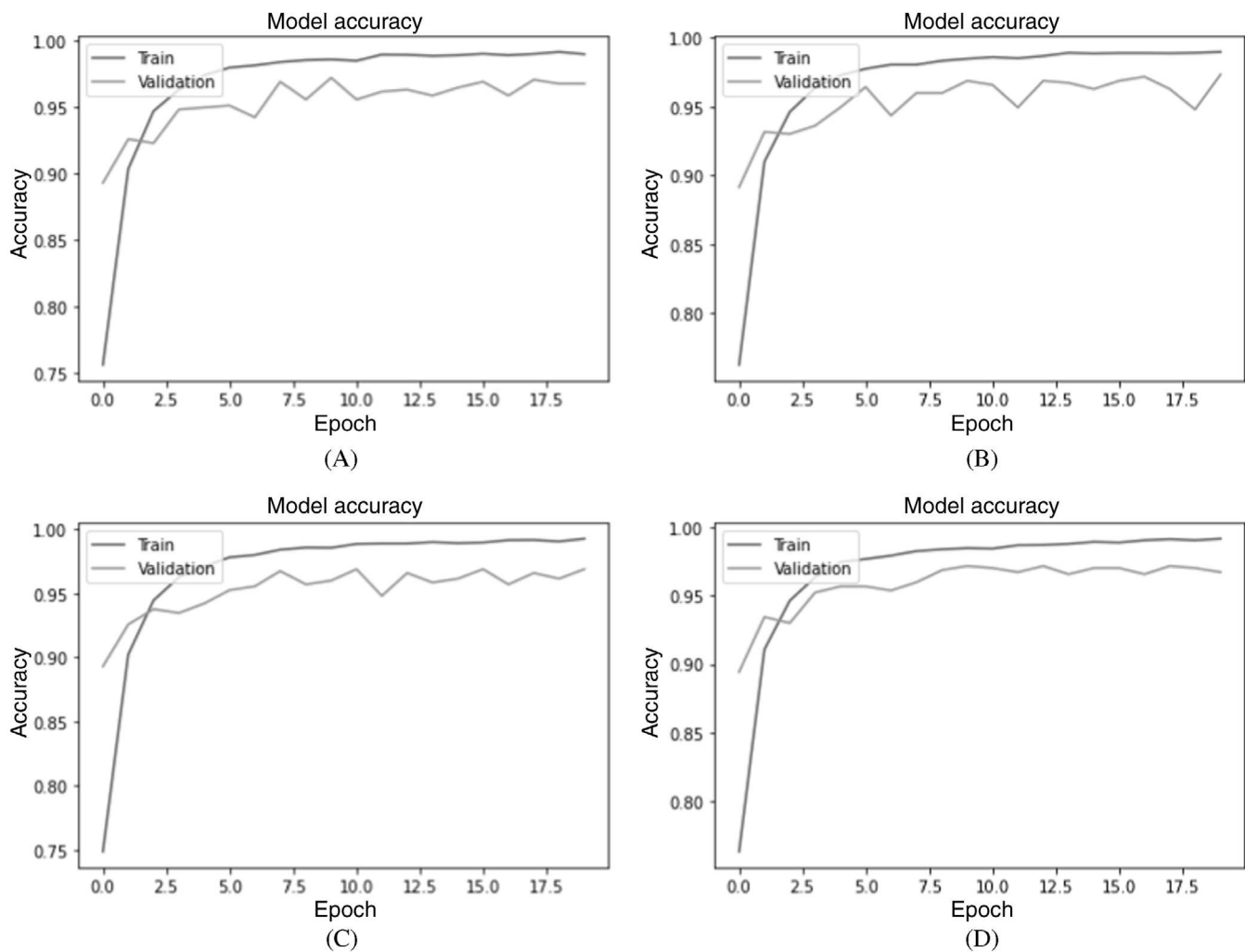


FIGURE 9 Illustration of the EfficientNetB0+ model results. (A) shows the squeeze and excitation attention module accuracy, (B) shows the self-attention module accuracy, (C) shows the convolutional block attention module (CBAM) attention module accuracy and (D) shows the dual attention module accuracy.

with attention modules for fingerprint pattern classification. The MobileNet and EfficientNet-B0 deep learning models were used as the base models. They were integrated with attention modules to form the MobileNet+ and EfficientNet-B0+ and their effect on the models' performance observed. The lightweight models conserve computational power, while the attention modules capture the contextual and precise information needed to classify fingerprint images accurately.

From the results obtained, the EfficientNet-B0+ model achieved a higher accuracy than the MobileNet+ model. The EfficientNet-B0+ model uses a swish activation function instead of the usual ReLU activation function. The swish activation function is smooth and does not move in one direction (nonmonotonic).⁴⁸ The swish activation function further reduces information loss, increasing its discriminative capacity and making the model perform better.

In the EfficientNet-B0+ model, it was observed that the dual attention module outperformed the other modules in all the metrics considered for evaluation. The dual attention module consists of the position and channel attention modules. The features are weighted across all positions, giving the module a global contextual view. The dual attention also selectively combines the global contexts with the spatial attention module. This can improve the intra-class variance and consistency. The channel attention has the final features of each channel as a sum of all channels and original features. This can help boost the feature discriminability due to the different dependencies between feature maps.

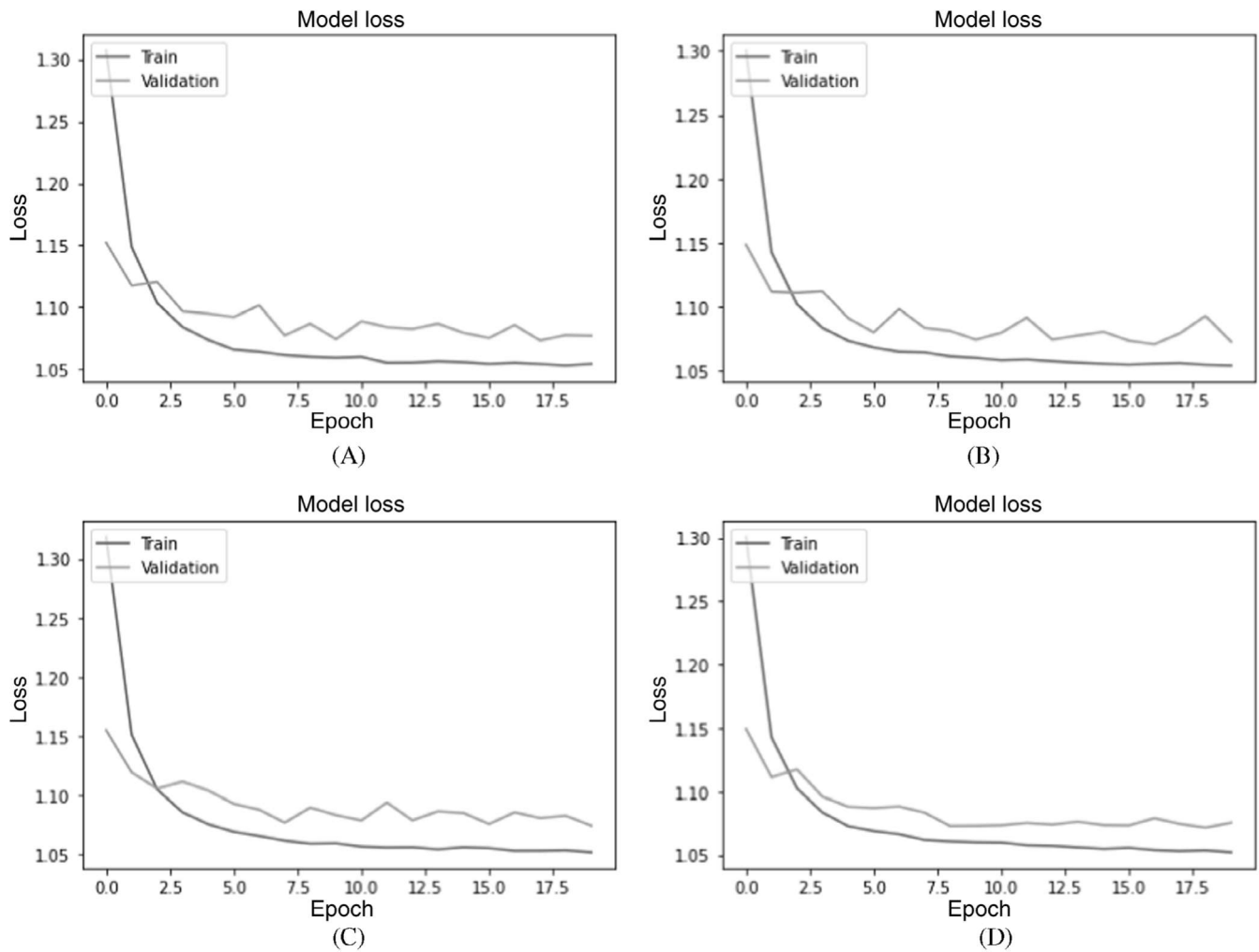


FIGURE 10 Illustration of the EfficientNetB0+ model results. (A) shows the squeeze and excitation attention module loss, (B) shows the self-attention module loss, (C) shows the convolutional block attention module (CBAM) attention module loss and (D) shows our attention module loss.

The SENet attention module had the lowest performance compared to the others. The module does not use spatial information and only relies on channel attention information. This limits the use of spatial information to capture more complex relationships in the data.

The self-attention and the CBAM attention modules were almost similar in their performances. The self-attention module had a higher network complexity and size cost than the dual-attention module. The module's architecture includes a ReLU activation layer in each convolutional branch, making it more costly.

To compare with other models, we consider the EfficientNet-B0+ model integrated with a dual attention module since it had better results than the MobileNet+ model. Table 5 compares our model with other studies. The comparison is based on the accuracy rate and the number of parameters. From the table, we can observe that the EfficientNet-B0+ model outperforms most of the existing models. The EfficientNet-B0+ model can be refined and deployed in edge devices. The study by Ametefe et al.⁴⁷ performed better than our proposed model but uses more training parameters than ours.

In conclusion, this paper proposes a fingerprint pattern classification model based on two pretrained lightweight deep learning models. Four attention modules were integrated into the two lightweight deep learning models, and their performance in fingerprint pattern classification was evaluated and analyzed. The efficientNetB0+ model integrated with dual attention was significantly the best among all the models. The experimental results demonstrate that lightweight deep learning models can achieve competitive performance in fingerprint pattern classification tasks when integrated

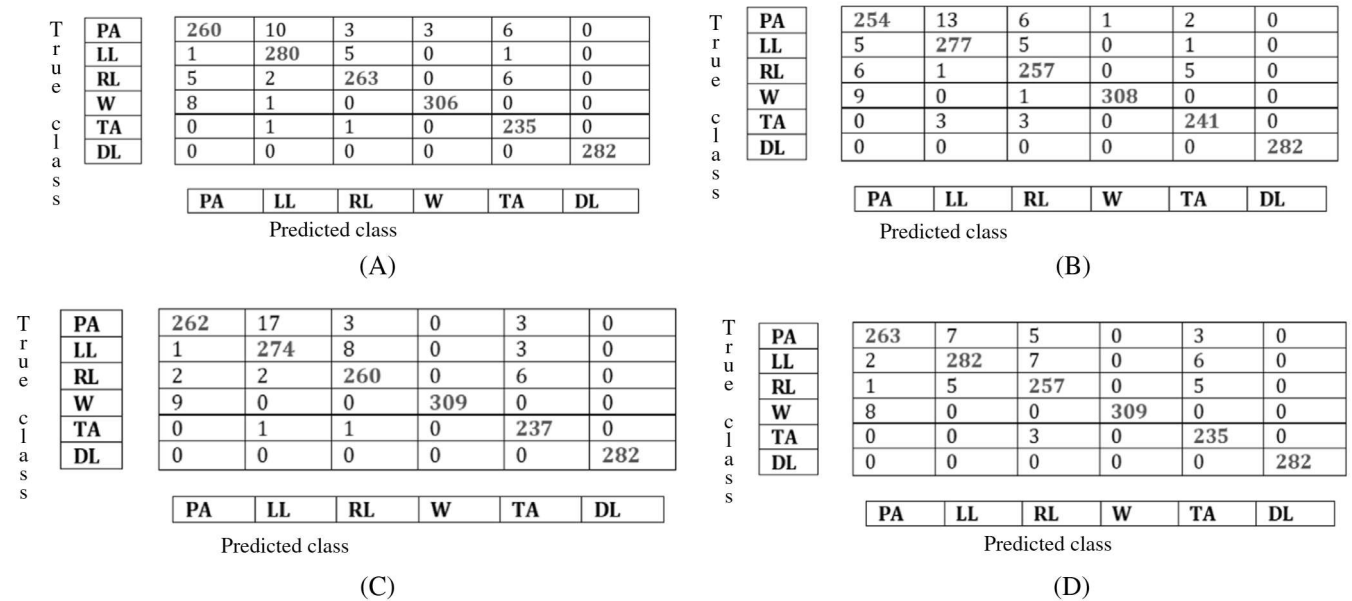


FIGURE 11 Illustration of the EfficientNetB0+ model results. (A) shows the squeeze and excitation attention module confusion matrix, (B) shows the self-attention module confusion matrix, (C) shows the convolutional block attention module (CBAM) attention module confusion matrix and (D) shows the dual attention module confusion matrix.

TABLE 5 Summary comparison with previous studies on fingerprint pattern classification.

Model	Dataset	Accuracy rate	Number of parameters
ResNet50 CNN ⁹	PolyU	95.7%	23M
stacked sparse autoencoder (SAE) neural network ⁴⁹	NIST-SD4	91.7%	23M
Deep learning with four CNN layers ⁵⁰	NIST-DB4	88.9%	23M
Bayesian Deep CNN ⁵¹	NIST-SD4 and FVC2004	92.2%	23M
Deep learning with data augmentation ⁴⁷	LivDet.Competition and NIST 301	VGG19, 97% DenseNet121, 97.8%	138M 7.2M
EfficientNet-B0+ model with dual attention module	LivDet Competition and NIST 301	97%	854,086

with attention modules. The efficientNetB0+ model demonstrates an acceptable performance with reduced complexity. We believe that the model has the potential for deployment in low-resource settings.

Future research can focus on developing attention modules that are interpretable and can provide insights into the decision-making process of deep learning models. Future research can also investigate attention modules that are dynamic and capable of adapting and changing over time during the learning process.

FUNDING INFORMATION

This research received no specific grant from any funding agency.

AUTHOR CONTRIBUTIONS

Esther Mukoya: conceptualization (equal); data curation (lead); formal analysis (lead); investigation (equal); methodology (lead); project administration (equal); resources (supporting); software (lead); validation (equal); visualization (equal); writing – original draft (lead). **Richard Rimiru:** conceptualization (supporting); investigation (equal); methodology (equal); project administration (lead); supervision (lead); validation (supporting); writing – original draft (supporting). **Michael Kimwele:** formal analysis (supporting); investigation (supporting); methodology (supporting); project administration (supporting); resources (lead); visualization (lead).

ACKNOWLEDGMENTS

We acknowledge the anonymous reviewers for reviewing the paper and providing feedback for improving it. We would also like to thank Isaac Nyabisa Oteyo for his helpful feedback and proofreading the paper.

CONFLICT OF INTEREST STATEMENT

The authors declare no conflict of interest.

DATA AVAILABILITY STATEMENT

The data that support the findings of this study are available from [NIST Special Database 301] and [LivDet Databases]. Restrictions apply to the availability of these data, which were used under license for this study. Data are derived from the following sources [https://www.nist.gov/itl/iad/image-group/nist-special-database-301] and [https://livdet.org/registration.php] with the permission of [NIST Special Database 301] and [LivDet Databases].

ORCID

Esther Mukoya  <https://orcid.org/0000-0003-1885-7136>

REFERENCES

- Militello C, Rundo L, Vitabile S, Conti V. Fingerprint classification based on deep learning approaches: experimental findings and comparisons. *Symmetry*. 2021;13(5):750.
- Nahar P, Tanwani S, Chaudhari NS. Fingerprint classification using deep neural network model resnet50. *Int J Res Anal Rev*. 2018;5(4):1521-1537.
- Pathak Y, Shukla PK, Tiwari A, Stalin S, Singh S. Deep transfer learning based classification model for COVID-19 disease. *Irbm*. 2022;43(2):87-92.
- Gao Y, Ma J, Zhao M, Liu W, Yuille AL. NDDR-CNN: Layerwise feature fusing in multi-task CNNs by neural discriminative dimensionality reduction. Paper presented at: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition. 2019:3205-3214.
- Lateef F, Ruichek Y. Survey on semantic segmentation using deep learning techniques. *Neurocomputing*. 2019;338:321-348.
- Wu F, Zhu J, Guo X. Fingerprint pattern identification and classification approach based on convolutional neural networks. *Neural Comput Appl*. 2020;32(10):5725-5734.
- Jian W, Zhou Y, Liu H. Lightweight convolutional neural network based on singularity ROI for fingerprint classification. *IEEE Access*. 2020;8:54554-54563.
- Pandya B, Cosma G, Alani AA, Taherkhani A, Bharadi V, McGinnity T. Fingerprint classification using a deep convolutional neural network. 2018 4th International Conference on Information Management (ICIM). IEEE; 2018:86-91.
- Minaee S, Azimi E, Abdolrashidi A. Fingernet: pushing the limits of fingerprint recognition using convolutional neural network. Fingernet. *arXiv Preprint arXiv:1907.12956*. 2019.
- He K, Zhang X, Ren S, Sun J. Deep residual learning for image recognition. *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*. IEEE; 2016:770-778.
- Gan J, Qi L, Bai Z, Xiang L. Fingerprint classification based on lightweight neural networks. *Chinese Conference on Biometric Recognition*. Springer; 2019:28-36.
- Saeed F, Hussain M, Aboalsamh HA. Automatic fingerprint classification using deep learning technology (DeepFKTNet). *Mathematics*. 2022;10(8):1285.
- Montalbo FJ. Truncating fined-tuned vision-based models to lightweight deployable diagnostic tools for SARS-CoV-2 infected chest X-rays and CT-scans. *Multimed Tools Appl*. 2022;81(12):16411-16439.
- Conti V, Militello C, Vitabile S, Sorbello F. Introducing pseudo-singularity points for efficient fingerprints classification and recognition. 2010 International Conference on Complex, Intelligent and Software Intensive Systems. IEEE; 2010:368-375.
- Militello C, Conti V, Sorbello F, Vitabile S. A novel embedded fingerprints authentication system based on singularity points. 2008 International Conference on Complex, Intelligent and Software Intensive Systems. IEEE; 2008:72-78.
- Saponara S, Elhanashi A, Zheng Q. Recreating fingerprint images by convolutional neural network autoencoder architecture. *IEEE Access*. 2021;9:147888-147899.
- Deshpande UU, Malemath VS, Patil SM, Chaugule SV. Automatic latent fingerprint identification system using scale and rotation invariant minutiae features. *Int J Inform Technol*. 2020;14(2):1-15.
- Ge S, Bai C, Liu Y, Liu Y, Zhao T. Deep and discriminative feature learning for fingerprint classification. 2017 3rd IEEE International Conference on Computer and Communications (ICCC). IEEE; 2017:1942-1946.
- Mnih V, Heess N, Graves A. Recurrent models of visual attention. *Adv Neural Inform Process Sys*. 2014;27.
- Huang H, Zhou P, Li Y, Sun F. A lightweight attention-based CNN model for efficient gait recognition with wearable IMU sensors. *Sensors*. 2021;21(8):2866.
- Ling H, Wu J, Huang J, Chen J, Li P. Attention-based convolutional neural network for deep face recognition. *Multimed Tools Appl*. 2020;79(9):5595-5616.

22. Zhang Z, Wang M. Convolutional neural network with convolutional block attention module for finger vein recognition. *arXiv Preprint arXiv:220206673*. 2022.
23. Michelsanti D, Ene AD, Guichi Y, Stef R, Nasrollahi K, Moeslund TB. Fast fingerprint classification with deep neural networks. *International Conference on Computer Vision Theory and Applications*. SCITEPRESS Digital Library; 2017:202-209.
24. Pan X, Ge C, Lu R, et al. On the integration of self-attention and convolution. *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*. IEEE; 2022:815-825.
25. Zhang J, Yu X, Lei X, Wu C. A novel deep LeNet-5 convolutional neural network model for image recognition. *Comput Sci Inf Syst*. 2022;19(3):36.
26. Hua X. DRN-SEAM: a deep residual network based on squeeze-and-excitation attention mechanism for motion recognition in education. *Comput Sci Inf Syst*. 2022;19(3):41.
27. Henry ER. *Classification and Uses of Fingerprints*. HM Stationery Office [printed by Harrison and sons, Limited]; 1922.
28. Ametefe DS, Sarnin SS, Ali DM, Zaheer MZ. Fingerprint liveness detection schemes: a review on presentation attack. *Comput Methods Biomech Biomed Eng Imaging Visual*. 2022;10(2):217-240.
29. Maltoni D, Maio D, Jain AK, Prabhakar S. *Handbook of Fingerprint Recognition*. Springer Science & Business Media; 2009.
30. LeCun Y, Boser B, Denker JS, et al. Backpropagation applied to handwritten zip code recognition. *Neural Comput*. 1989;1(4):541-551.
31. Michele A, Colin V, Santika DD. Mobilenet convolutional neural networks and support vector machines for palmprint recognition. *Proc Comput Sci*. 2019;157:110-117.
32. Hsiao CS, Fan CP. EfficientNet based iris biometric recognition methods with pupil positioning by U-net. *2021 3rd International Conference on Computer Communication and the Internet (ICCCI)*. IEEE; 2021:1-5.
33. Tan M, Le Q. Efficientnet: rethinking model scaling for convolutional neural networks. *International Conference on Machine Learning*. PMLR; 2019:6105-6114.
34. Huang C, Wang W, Zhang X, Wang SH, Zhang YD. Tuberculosis diagnosis using deep transferred EfficientNet. *IEEE/ACM Trans Comput Biol Bioinform*. 2022;20(5):2639-2646.
35. Wang F, Jiang M, Qian C, et al. Residual attention network for image classification. *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*. IEEE; 2017:3156-3164.
36. Woo S, Park J, Lee JY, Kweon IS. Cbam: convolutional block attention module. *Proceedings of the European Conference on Computer Vision (ECCV)*. Springer; 2018:3-19.
37. Hu J, Shen L, Sun G. Squeeze-and-excitation networks. *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*. IEEE; 2018:7132-7141.
38. Ling H, Wu J, Wu L, Huang J, Chen J, Li P. Self residual attention network for deep face recognition. *IEEE Access*. 2019;7:55159-55168.
39. Fu J, Liu J, Tian H, et al. Dual attention network for scene segmentation. *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*. IEEE; 2019:3146-3154.
40. Wang SH, Fernandes SL, Zhu Z, Zhang YD. AVNC: attention-based VGG-style network for COVID-19 diagnosis by CBAM. *IEEE Sensors J*. 2021;22(18):17431-17438.
41. Zhang YD, Zhang Z, Zhang X, Wang SH. MIDCAN: a multiple input deep convolutional attention network for Covid-19 diagnosis based on chest CT and chest X-ray. *Pattern Recogn Lett*. 2021;150:8-16.
42. Zhou Y, Zhu Z, Zhong Z. Learning specialized activation functions with the piecewise linear unit. *Proceedings of the IEEE/CVF International Conference on Computer Vision*. IEEE; 2021:12095-12104.
43. Fiumara GP, Flanagan PA, Grantham JD, et al. *NIST Special Database 301: Nail to Nail Fingerprint Challenge Dry Run*. NIST; 2018.
44. Gupta P, Tiwari K, Arora G. Fingerprint indexing schemes—a survey. *Neurocomputing*. 2019;335:352-365.
45. Iwamoto M. On the classification of fingerprints. *J Anthropol Soc Nippon*. 1962;69(3-4):101-117.
46. Jongh DA, Lubach AR, Lie Kwie SL, Alberink I. Measuring the rarity of fingerprint patterns in the Dutch population using an extended classification set. *J Forensic Sci*. 2019;64(1):108-119.
47. Ametefe DS, Sarnin SS, Ali DM, Muhammad ZZ. Fingerprint pattern classification using deep transfer learning and data augmentation. *Vis Comput*. 2022;39(4):1-14.
48. Ramachandran P, Zoph B, Le QV. Searching for activation functions. *arXiv Preprint arXiv:171005941*. 2017.
49. Wang R, Han C, Guo T. A novel fingerprint classification method based on deep learning. *2016 23rd International Conference on Pattern Recognition (ICPR)*. IEEE; 2016:931-936.
50. Peralta D, Triguero I, García S, Saeys Y, Benitez JM, Herrera F. On the use of convolutional neural networks for robust classification of multiple fingerprint captures. *Int J Intell Syst*. 2018;33(1):213-230.
51. Zia T, Ghafoor M, Tariq SA, Taj IA. Robust fingerprint classification with Bayesian convolutional networks. *IET Image Process*. 2019;13(8):1280-1288.

How to cite this article: Mukoya E, Rimiru R, Kimwele M. Enhanced fingerprint pattern classification: Integrating attention modules with lightweight deep learning models. *Engineering Reports*. 2024;6(11):e12897. doi: 10.1002/eng2.12897

APPENDIX A. IMPLEMENTATION OF THE EFFICIENTNET-B0+ MODEL DEFINITION

Figure A1 shows the implementation of the EfficientNet-B0+ model. The details of the model were explained in Section 3. Note that the model builds on the EfficientNet-B0 as the base model. Generally, this code demonstrates our proposed efficientB0+ model. We show the different attention modules after block seven. The forward method describes how input data flows through the model's layers.

```
class Model(nn.Module):
    def __init__(self, backbone = "efficientnet", flag = 2):
        super(Model, self).__init__()
        #EfficientNet Backbone
        self.efficientnet = EfficientNet.from_pretrained('efficientnet-b0')
        #Squeeze and Excitation
        self.se_layer = SELayer()
        #Self Attention
        self.self_attention = self_attention()
        #CBAM
        self.CBAM = CBAM()
        #Dual Attention
        self.dual = Dual_Attention()

        self.avg_pool = nn.AdaptiveAvgPool2d(1)
        self.linear1 = nn.Linear(128, 6)
        self.softmax = nn.Softmax(dim=1)
        self.flag = flag
        self.backbone = backbone

    def forward(self, x):
        x1 = self.efficientnet._conv_stem(x)
        out = self.efficientnet._bn0(x1)
        out = self.efficientnet._blocks[0](out)
        out = self.efficientnet._blocks[1](out)
        out = self.efficientnet._blocks[2](out)
        out = self.efficientnet._blocks[3](out)
        out = self.efficientnet._blocks[4](out)
        out = self.efficientnet._blocks[5](out)
        out = self.efficientnet._blocks[6](out)
        out = self.efficientnet._blocks[7](out)

        if self.flag == 1:
            out = self.self_attention(out)
        elif self.flag == 2:
            out = self.se_layer(out)
        elif self.flag == 3:
            out = self.CBAM(out)
        elif self.flag == 4:
            out = self.dual(out)

        out = self.efficientnet._blocks[8](out)
        out = self.efficientnet._blocks[9](out)
        out = self.efficientnet._blocks[10](out)
        out = self.efficientnet._blocks[11](out)
        out = self.efficientnet._blocks[12](out)
        out = self.efficientnet._blocks[13](out)
        out = self.efficientnet._blocks[14](out)
        out = self.efficientnet._blocks[15](out)
        out = self.efficientnet._blocks[16](out)
        b,c,_,_ = out.size()
        out = self.adaptiveAvgPool(out).view(b,c)
        out = self.linear1(out)
        out = self.softmax(out)

        return out
model = Model(flag = 2).cuda()
```

FIGURE A1 Implementation of the EfficientNet-B0+ model.

```

from torchsummary import summary
#from delivery import Model
#from efficientnet_lightweight import Model for efficientNetB0 Model
#from MobileNet_lightweight import Model for MobileNet Model

torch.cuda.empty_cache()
seeding(42)
model = Model(flag = 2)
#device = torch.device('cuda')
#model = model.to(device)
device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')

t_loss = []
v_loss = []
t_acc = []
v_acc = []

loss = nn.CrossEntropyLoss()
optimizer = torch.optim.Adam(model.parameters(), lr = 1e-4)

batch_size = 4
num_epochs = 20
checkpoint_path = "model_4"

dataset_train = MyDataset(X_train, y_train)
dataset_test = MyDataset(X_test, y_test)

dataloader = DataLoader(dataset_train, batch_size = batch_size)
dataloader_test = DataLoader(dataset_test, batch_size = batch_size)

best_valid_loss = float('inf')
for epoch in range(num_epochs):

    train_loss, acc_t = train(model, dataloader, optimizer, loss, device)
    valid_loss, acc_v = evaluate(model, valid_loader, loss, device)

    t_loss.append(train_loss)
    v_loss.append(valid_loss)
    t_acc.append(acc_t)
    v_acc.append(acc_v)

    if valid_loss < best_valid_loss:
        best_valid_loss = valid_loss
        torch.save(model.state_dict(), checkpoint_path)

    print("Epoch: ", epoch + 1)
    print("Train Loss: ", train_loss)
    print("Validation Loss: ", valid_loss)
    print("Train Accuracy:", acc_t)
    print("Validation Accuracy", acc_v)

import matplotlib.pyplot as plt
plt.plot(t_loss)
plt.plot(v_loss)
plt.title('model loss')
plt.ylabel('loss')
plt.xlabel('epoch')
plt.legend(['Train', 'Validation'], loc='upper left')

plt.show()

plt.plot(t_acc)
plt.plot(v_acc)
plt.title('model accuracy')
plt.ylabel('accuracy')
plt.xlabel('epoch')
plt.legend(['Train', 'Validation'], loc='upper left')
plt.show()

```

FIGURE B1 Implementation for plotting the evaluation results for the models.

Lines 1 and 2 initialize the model and give default values. Line 5 performs the backbone initialization using the pre-trained model. Lines 6–13 initiate the different attention modules. Lines 15–17 initialize the adaptive average pooling, the linear layer with an input of 128 and output of six classes, and the softmax. The model's forward pass is defined from lines 21–51. We first define the initial blocks of the model, then, depending on a selected flag, apply an attention module and continue with the remaining blocks of the model. We then end with global average pooling and linear operations on lines 52–54.

APPENDIX B. IMPLEMENTATION FOR PRODUCING THE EVALUATION RESULTS

Figure B1 shows the implementation for printing the evaluation results for the model. This code trains the model using the specified architecture, in this case, efficientNetB0+ on the dataset. We initiate the model and define the training parameters on lines 12–23. We then create and load the datasets for training and testing on lines 25–29. The training loop records training and validation losses and accuracies for each epoch on lines 31–50. The model with the lowest validation loss is saved. We visualize the loss and accuracy plots using Matplotlib on lines 52–68.